Mastering Diffusion

Week 10 – Diffusion language models 3

Yejoon Lee

AIDAS Lab
Seoul National University
leeyejoon@snu.ac.kr

AIDAS Lab

Table of contents

A. Sampling
   1. Basic DLM sampling (non-AR)
   2. Semi-AR sampling & approximate KV caching
   3. Block diffusion (AR-diffusion hybrid)

B. Post-training
   1. Diffusion-of-thought
   2. GRPO for DLM (d1)

DLM Sampling

Review: Masked Diffusion Language Model (MDLM)

MDLM loss is a NLL on masked tokens

$$\mathcal{L}(\theta) \triangleq -\mathbb{E}_{t,x_0,x_t}\left[\frac{1}{t}\sum_{i=1}^{L}\mathbf{1}[x_t^i=\mathbf{M}]\log p_\theta(x_0^i|x_t)\right]$$

This loss is an upper bound of NLL of data, just like the generative models we've seen

$$-\mathbb{E}_{p_{\text{data}}(x_0)}\left[\log p_\theta(x_0)\right] \leq \mathcal{L}(\theta),$$

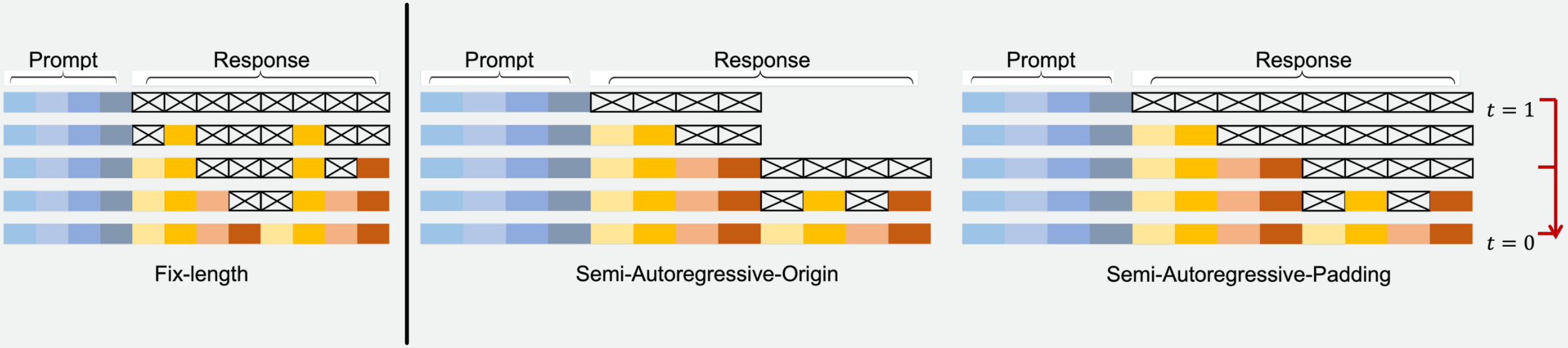
Throughout this talk, Diffusion Language Model(DLM) means MDLM

Nie et al., 2025, Large Language Diffusion Models.

Three flavors of sampling DLM

In DLM, we can define **“block”** and the tokens are sampled block-by-block

LLaDA mentions three flavors of DLM sampling.



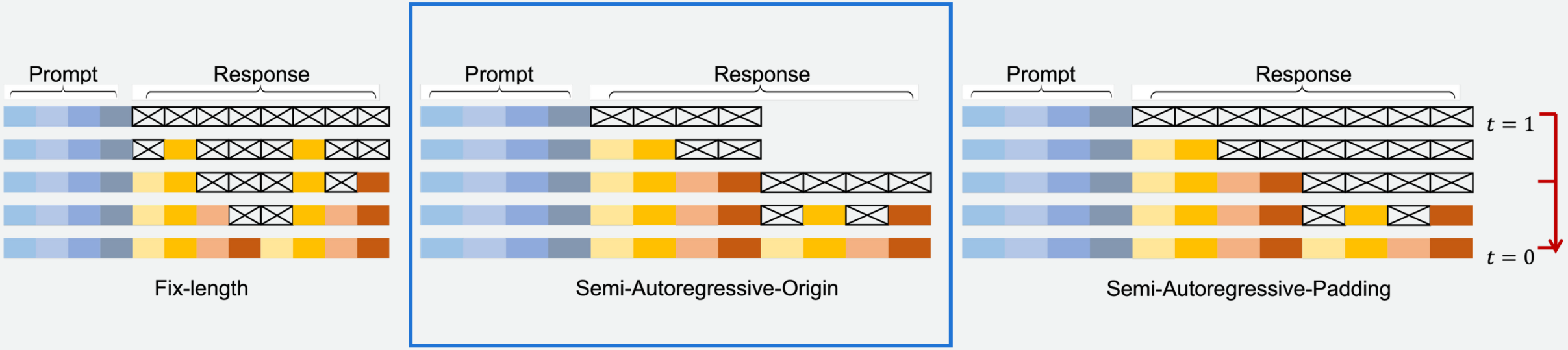
It may seem that the right two are more similar than the other…

https://github.com/ML-GSAI/LLaDA/blob/main/GUIDELINES.md

Three flavors of sampling DLM

In DLM, we can define **“block”** and the tokens are sampled block-by-block

LLaDA mentions three flavors of DLM sampling.

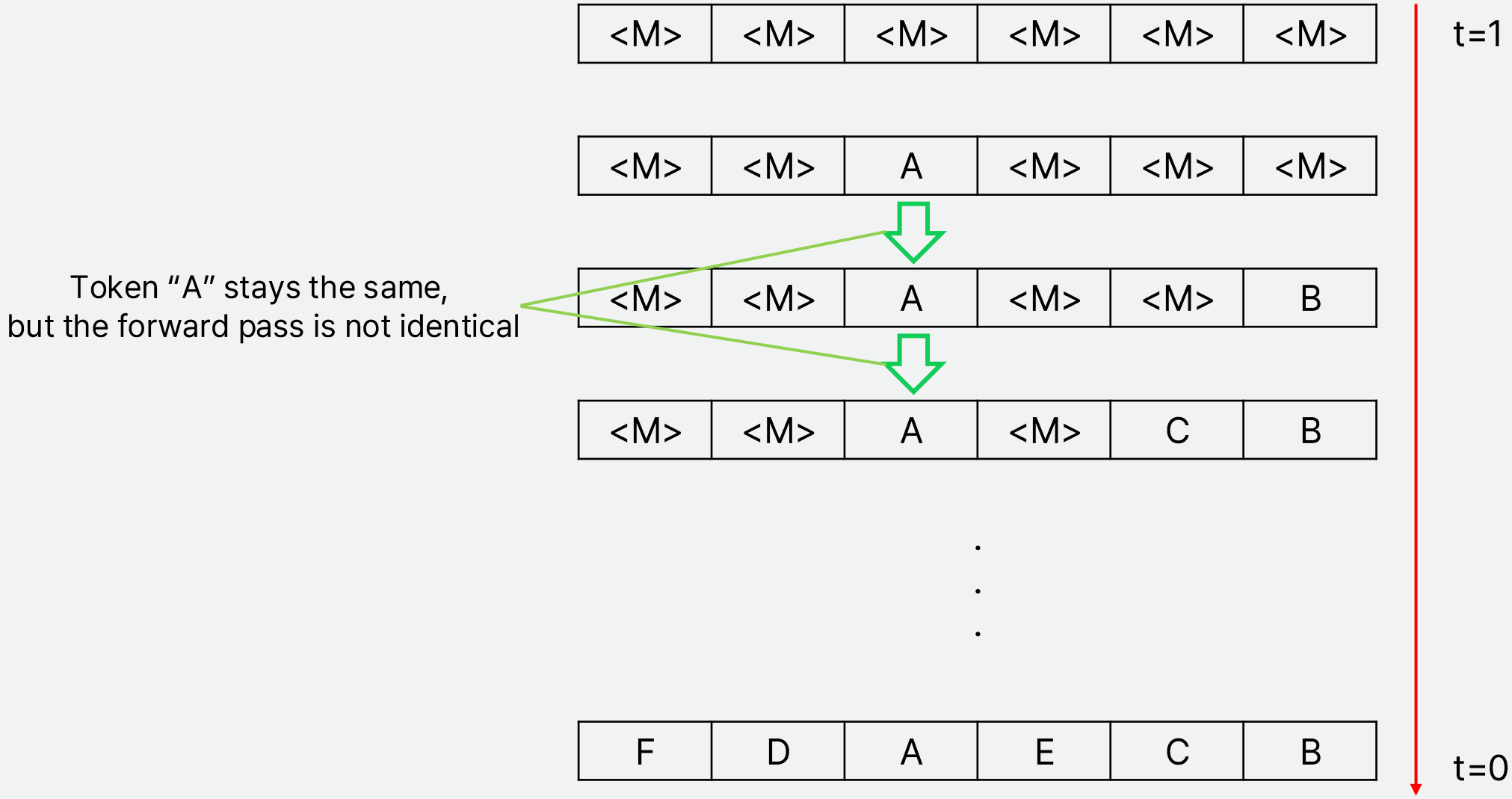


By the end you will know that the middle one is the distinct one

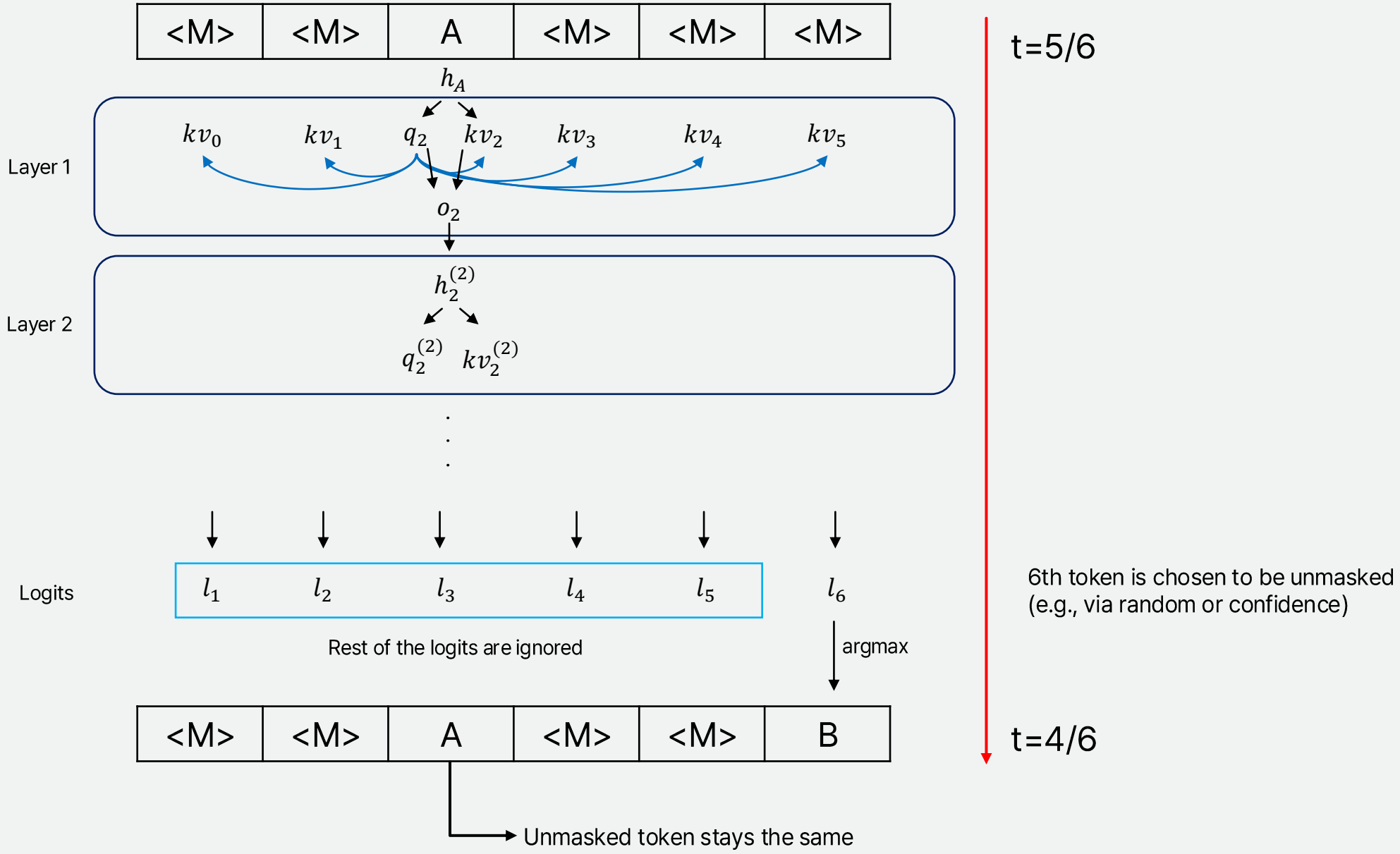
https://github.com/ML-GSAI/LLaDA/blob/main/GUIDELINES.md

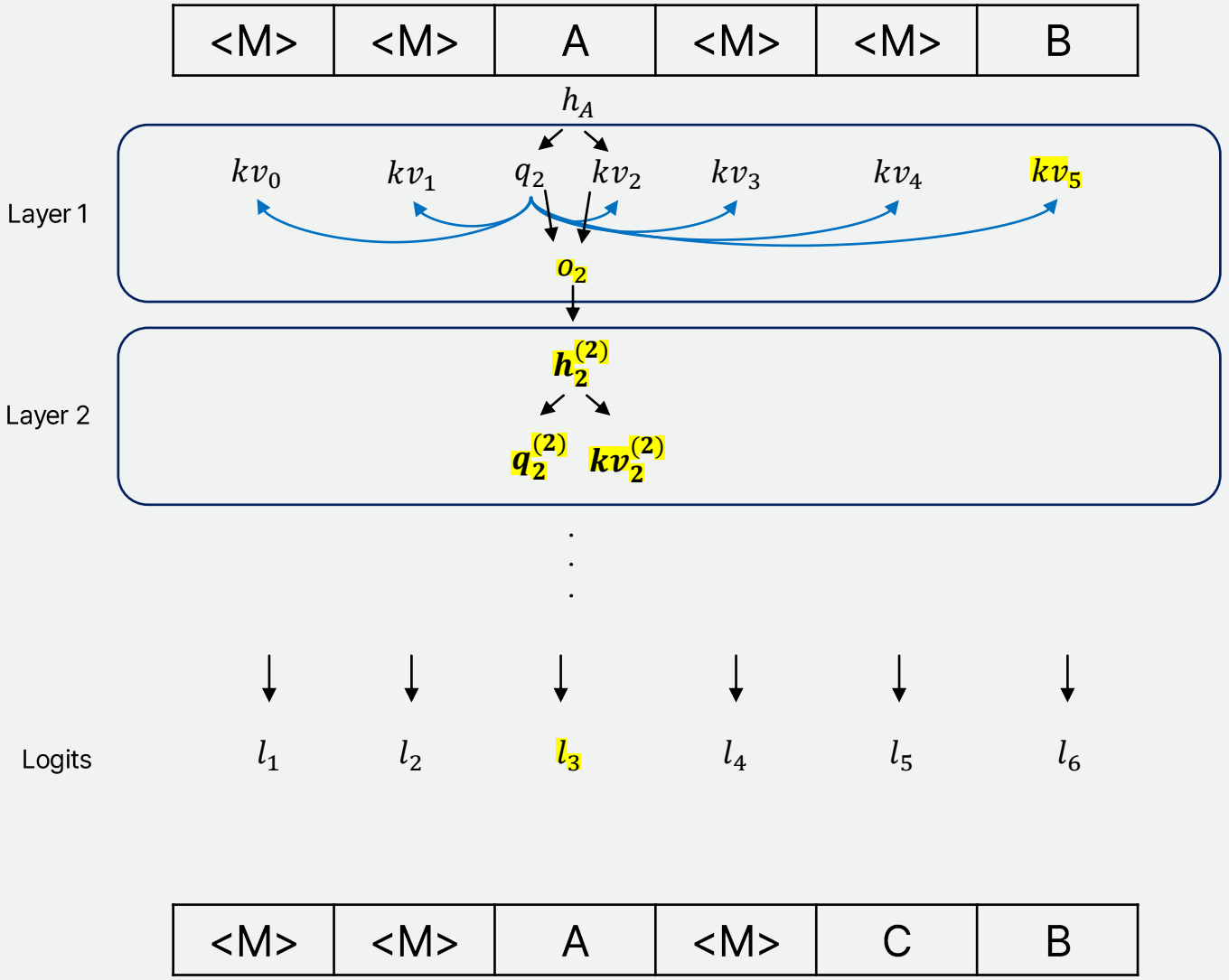
Table of contents

A. Sampling
    1. Basic DLM sampling (non-AR)
    2. Semi-AR sampling & approximate KV caching
    3. Block diffusion (AR-diffusion hybrid)


Token “A” stays the same,
but the forward pass is not identical
<M> <M> <M> <M> <M> <M>
t=1
<M> <M> A <M> <M> <M>
<M> <M> A <M> <M> B
<M> <M> A <M> C B
F D A E C B
t=0

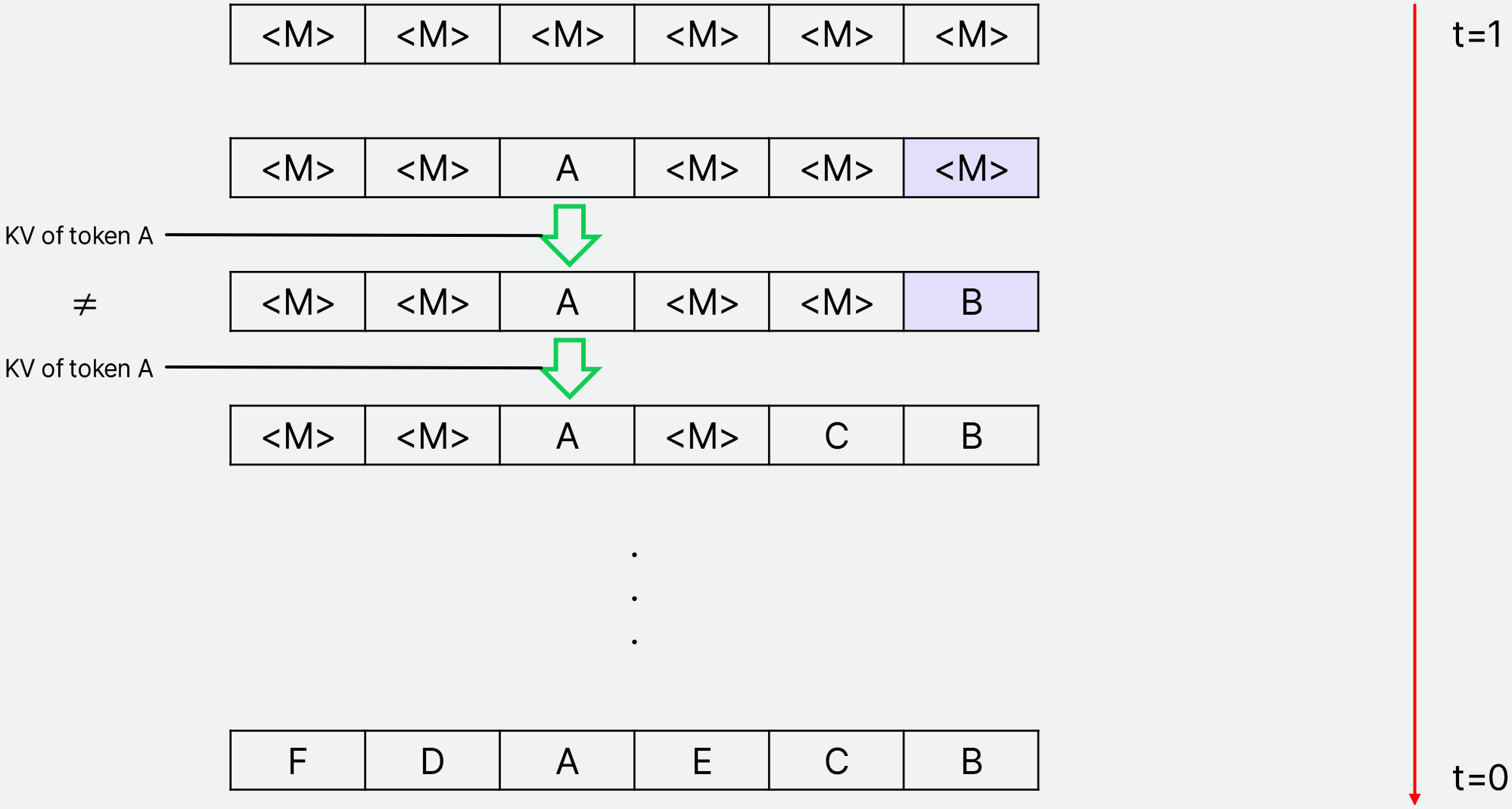
Inside the forward pass,

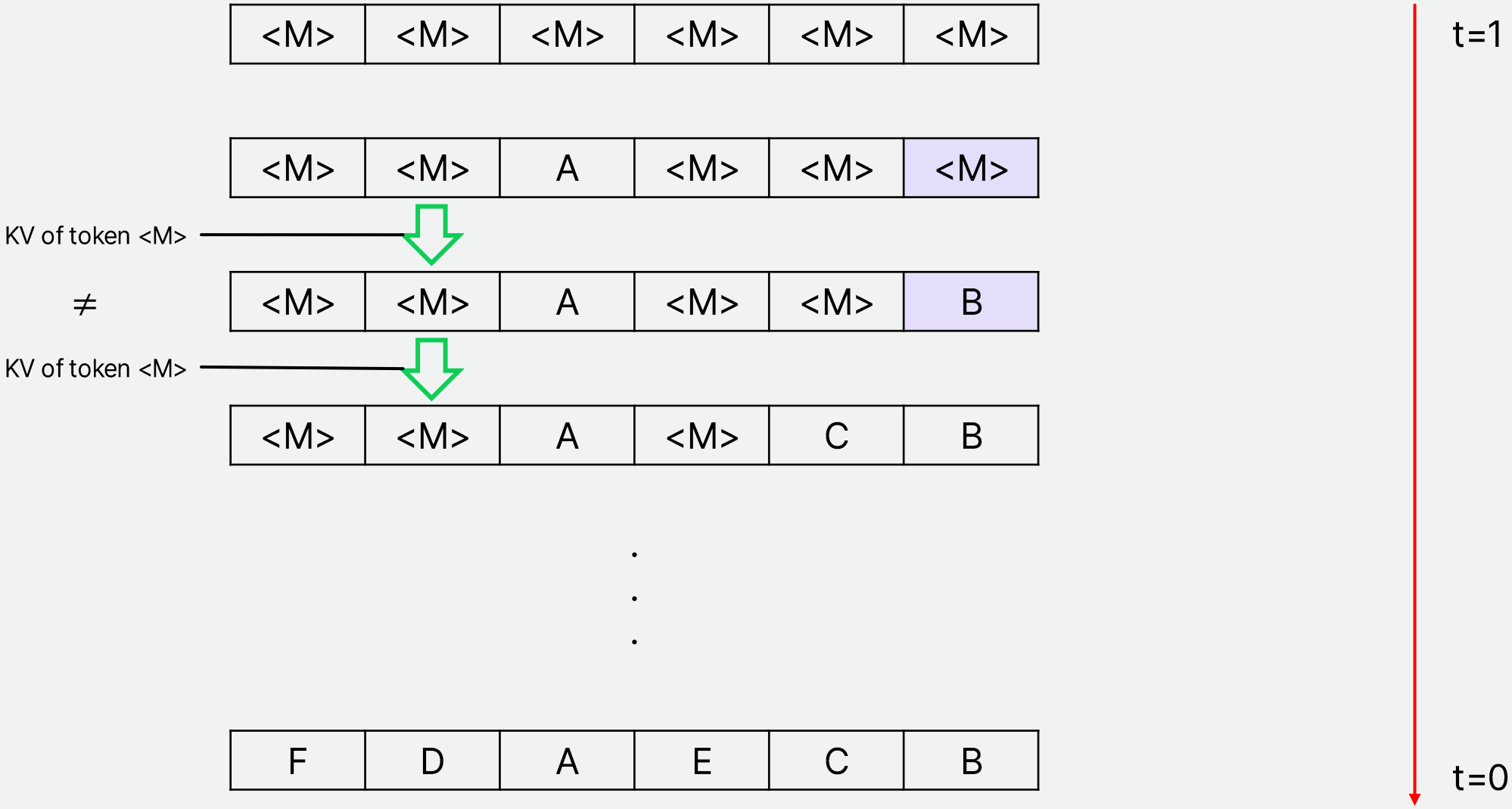


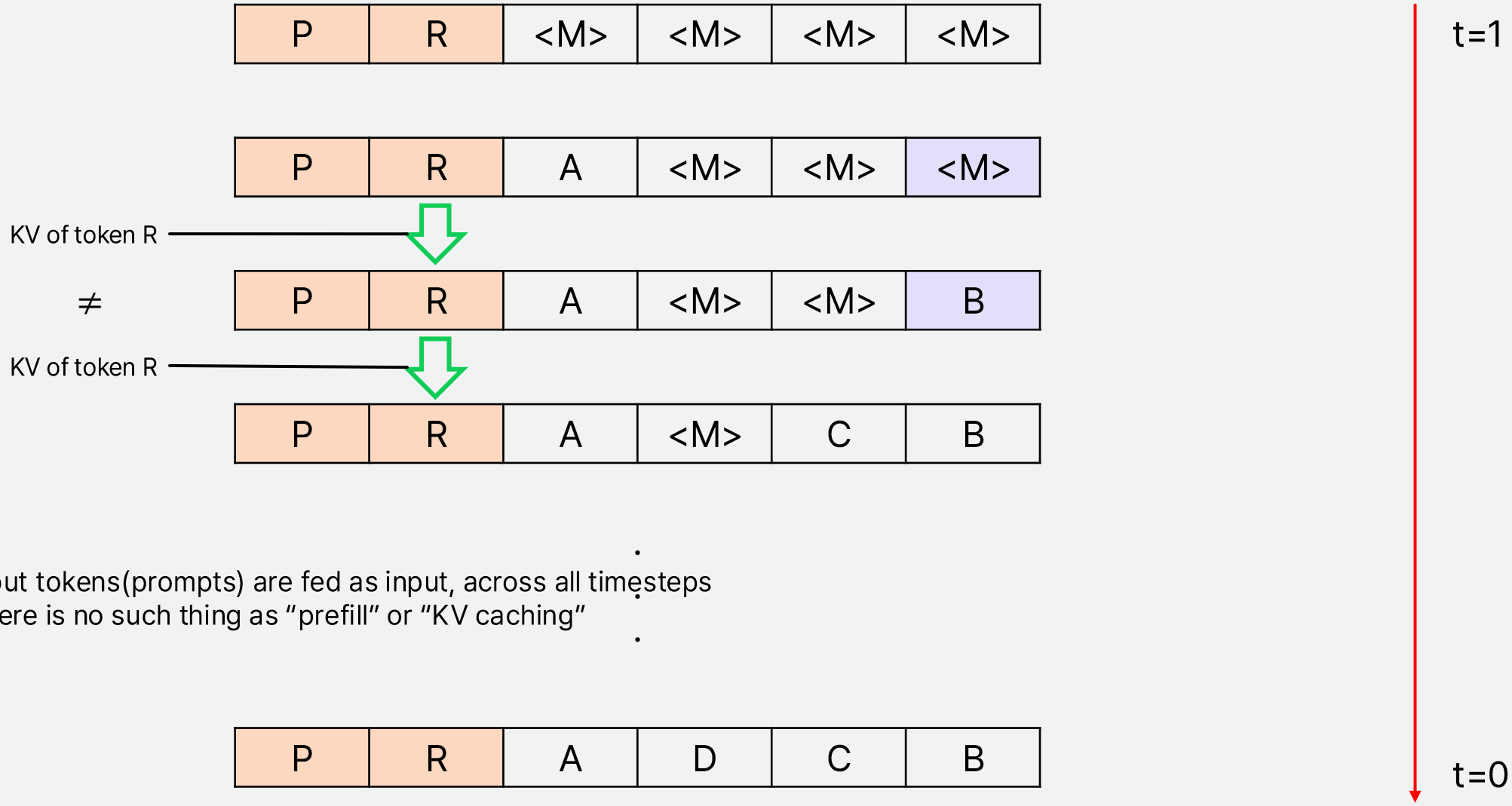


KV of 6th token has changed from <M> to B

After attention with kv_5, all the proceeding results change


<M> <M> <M> <M> <M> <M>
<M> <M> A <M> <M> <M>
KV of token A
≠
<M> <M> A <M> <M> B
KV of token A
<M> <M> A <M> C B
F D A E C B
t=1
t=0


<M> <M> <M> <M> <M> <M>
<M> <M> A <M> <M> <M>
KV of token <M>
≠
<M> <M> A <M> <M> B
KV of token <M>
<M> <M> A <M> C B
F D A E C B
t=1
t=0

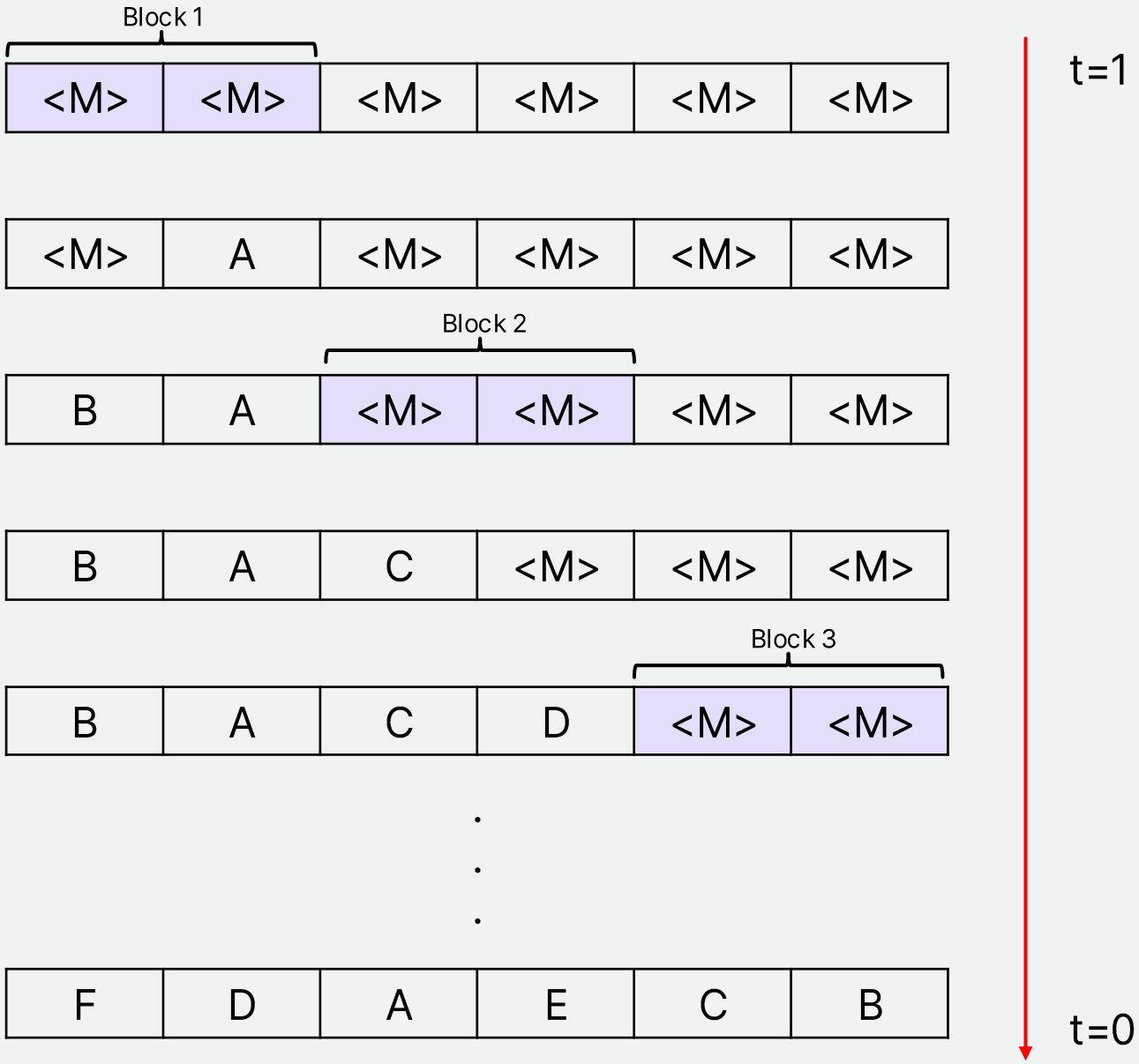


Input tokens(prompts) are fed as input, across all timesteps
There is no such thing as “prefill” or “KV caching”

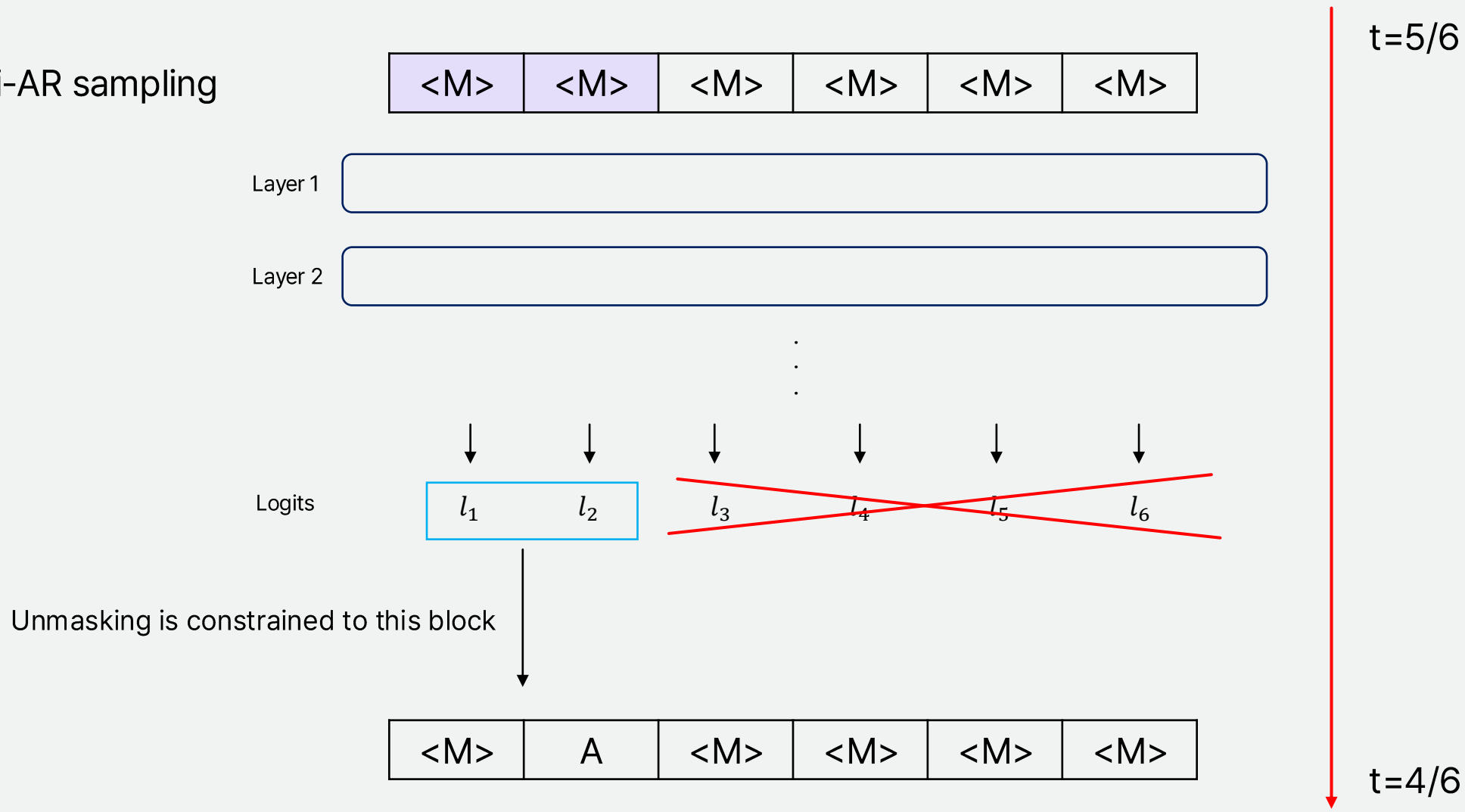
Semi-AR sampling
& approximate KV caching

Introducing semi-AR sampling:

Only the tokens in current block can be unmasked.



Semi-AR sampling



But why semi-AR sampling for DLMs?

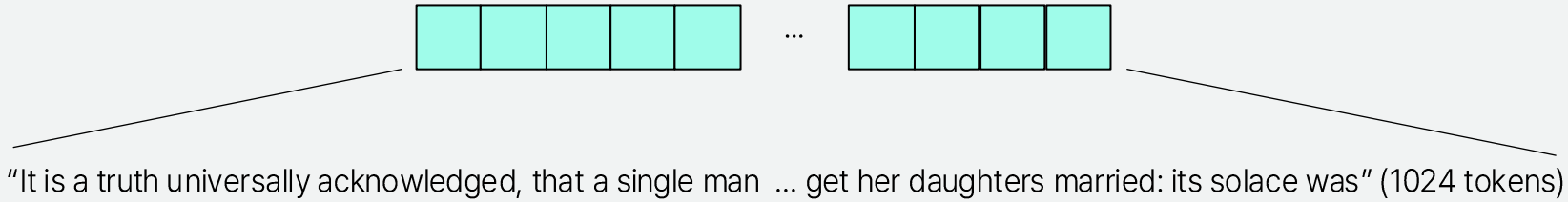
- For better variable-length generation
  - LLaDA (Nie et al., 2025)

- For efficient sampling
  - Fast-dLLM (Wu et al., 2025)

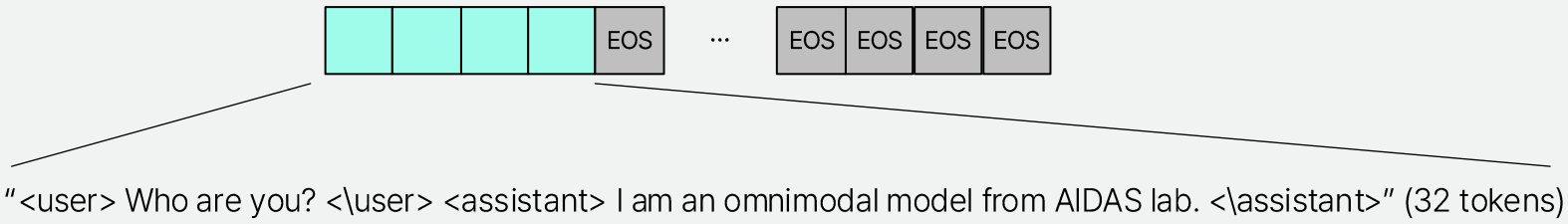
Nie et al., 2025, Large Language Diffusion Models. Wu et al., 2025, Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding

For better variable-length generation

- In pre-training, sequence is (almost) always filled up to max seq length

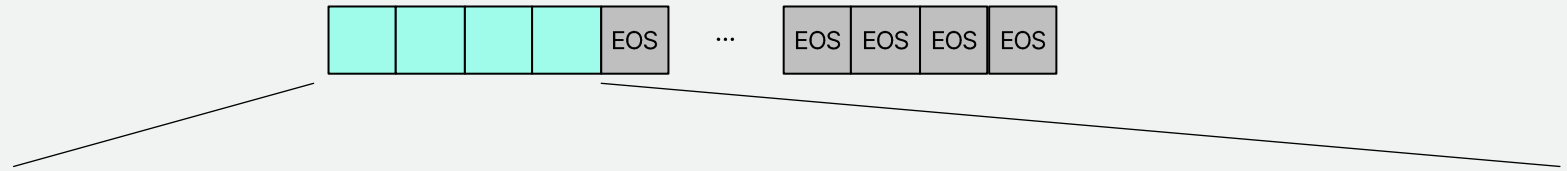


- In SFT, sequences have different lengths



For better variable-length generation

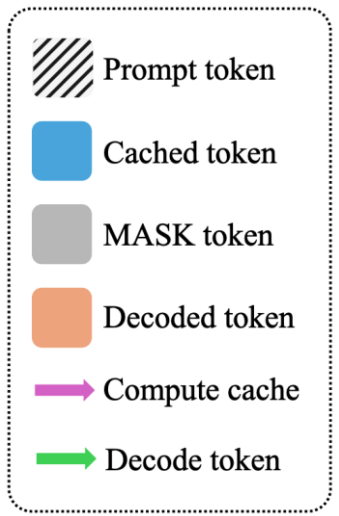
- During SFT, model is likely trained on a lot of EOS tokens
- LLaDA reports that
  - For base model, fixed-length and semi-AR achieved similar results
  - **Fixed-length sampling + confidence-based unmaksing** led to unusually **high proportion of EOS** tokens (this issue did not happen in random unmasking)
  - Semi-AR + confidence-based unmaksing > Fixed-length + random unmasking

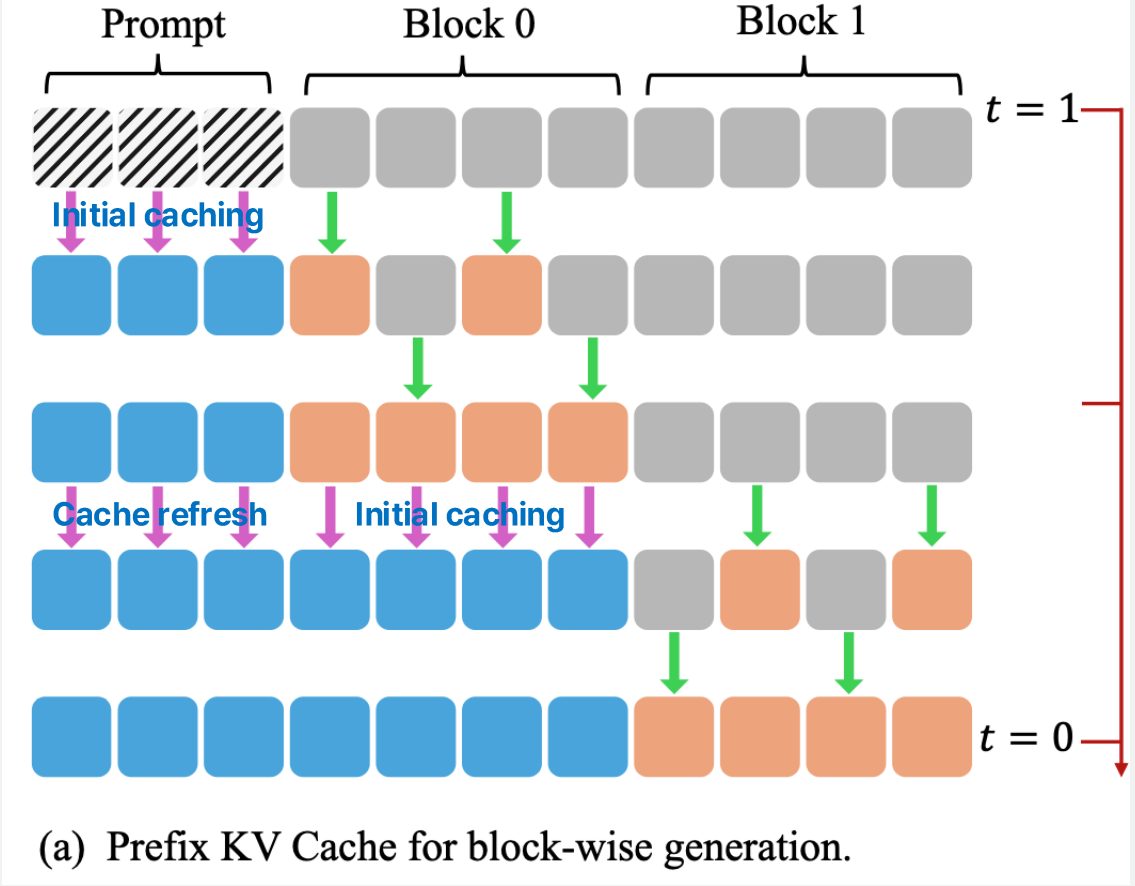


Approximate KV caching in Fast-dLLM

- Fast-dLLM speeds up DLM inference via
  - Approximate KV caching
  - Confidence-aware parallel decoding

- For approximate KV caching, Fast-dLLM adopts semi-AR sampling

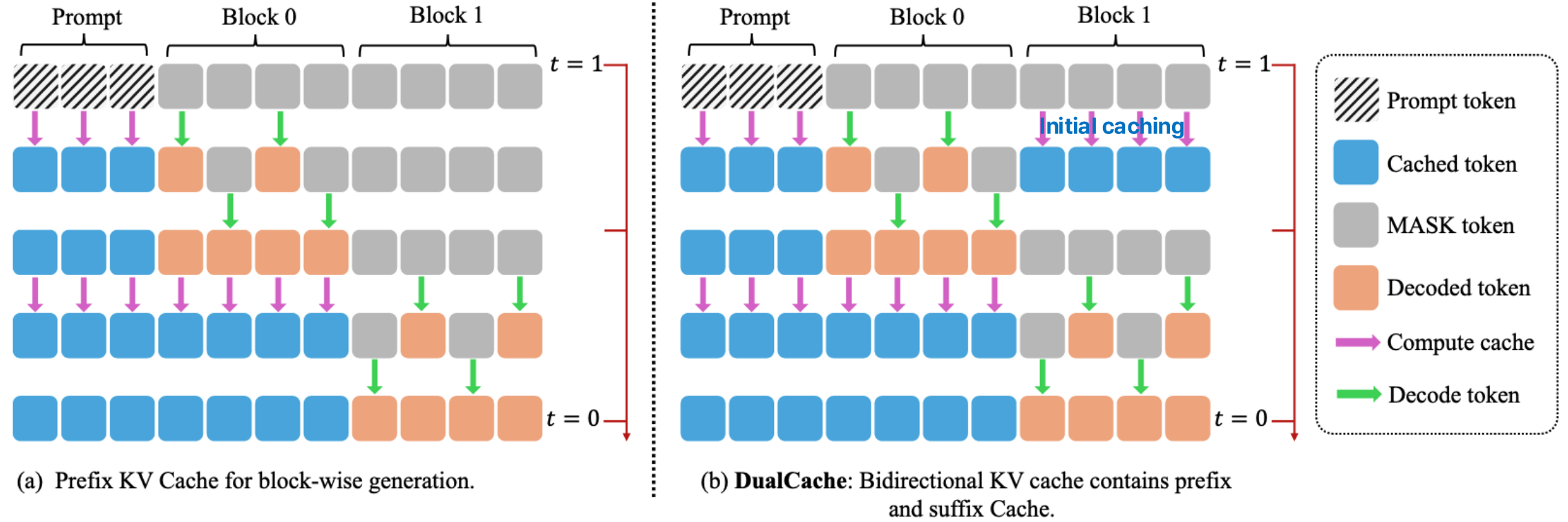




Wu et al., 2025, Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding

Approximate KV caching in Fast-dLLM

Same could be done for mask tokens



(a) Prefix KV Cache for block-wise generation.

(b) **DualCache**: Bidirectional KV cache contains prefix and suffix Cache.

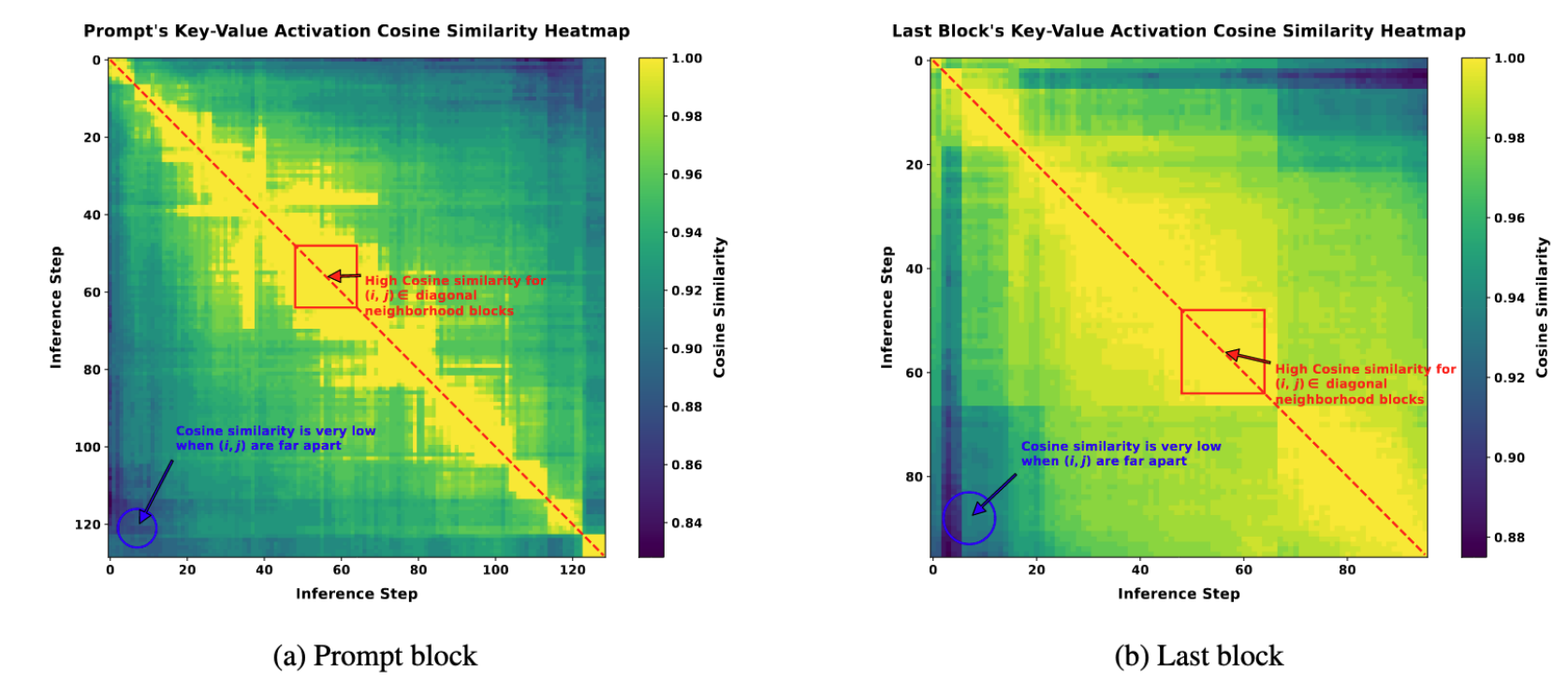
Wu et al., 2025, Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding

Approximate KV caching in DLMs

- Exact KV caching (AR-like) is impossible in DLMs
- But what is the rationale behind KV caching? Can we really skip the computation?

Approximate KV caching in DLMs

- Tokens of prompts do not change throughout all the timesteps ($0 \leq t \leq 128$)
- Mask tokens of the last block do not change throughout all the timesteps except for the last block ($0 \leq t \leq 96$)
- High cosine similarity of KV across the timesteps



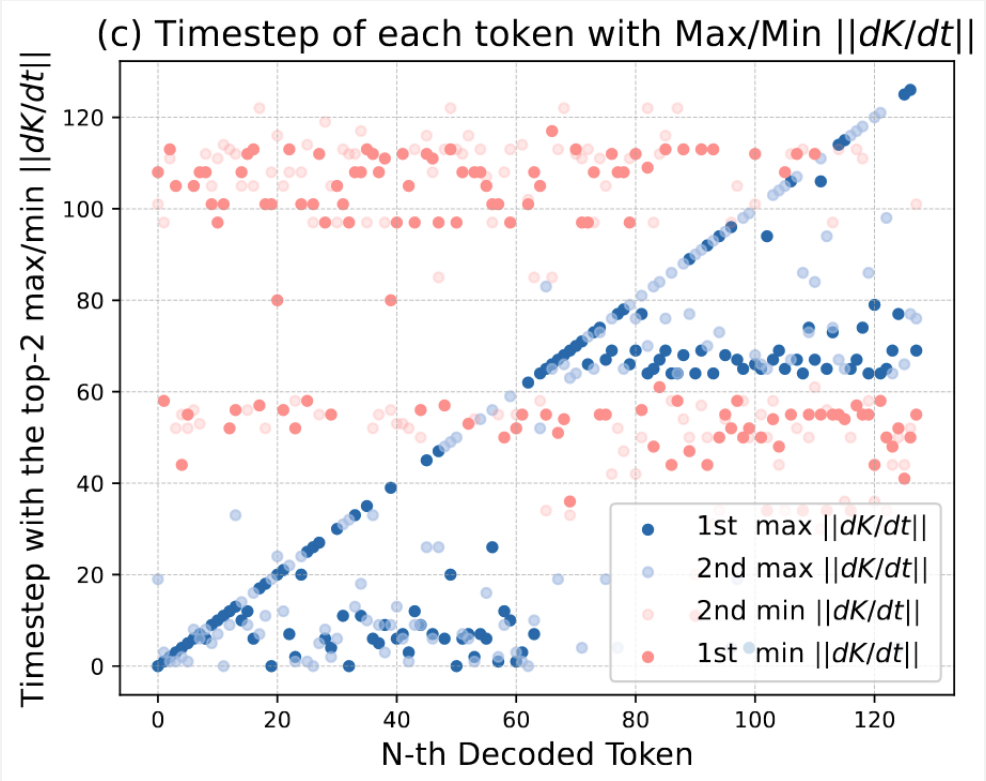
(a) Prompt block (b) Last block

LLaDA-Instruct,
averaged over all tokens of
prompt/last block

Wu et al., 2025, Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding

Approximate KV caching in DLMs

- Journey of every decoded token of MDLM:
  - <M> → <M> → … → <M> → 47 → 47 → … → 47
- KV changes most significantly for timesteps
  - where the token is decoded(unmasked)
  - in the beginning of each block
- KV changes the least for timesteps
  - in the end of each block



LLaDA, gen_len = 128, timestep = 128, block size= 64

Ma et al., 2025, dKV-Cache: The Cache for Diffusion Language Models

Long context + DLM = 🐌

- AR LM decoding time complexity
  - Transformer with KV caching: O(n)
  - => O(n^2)

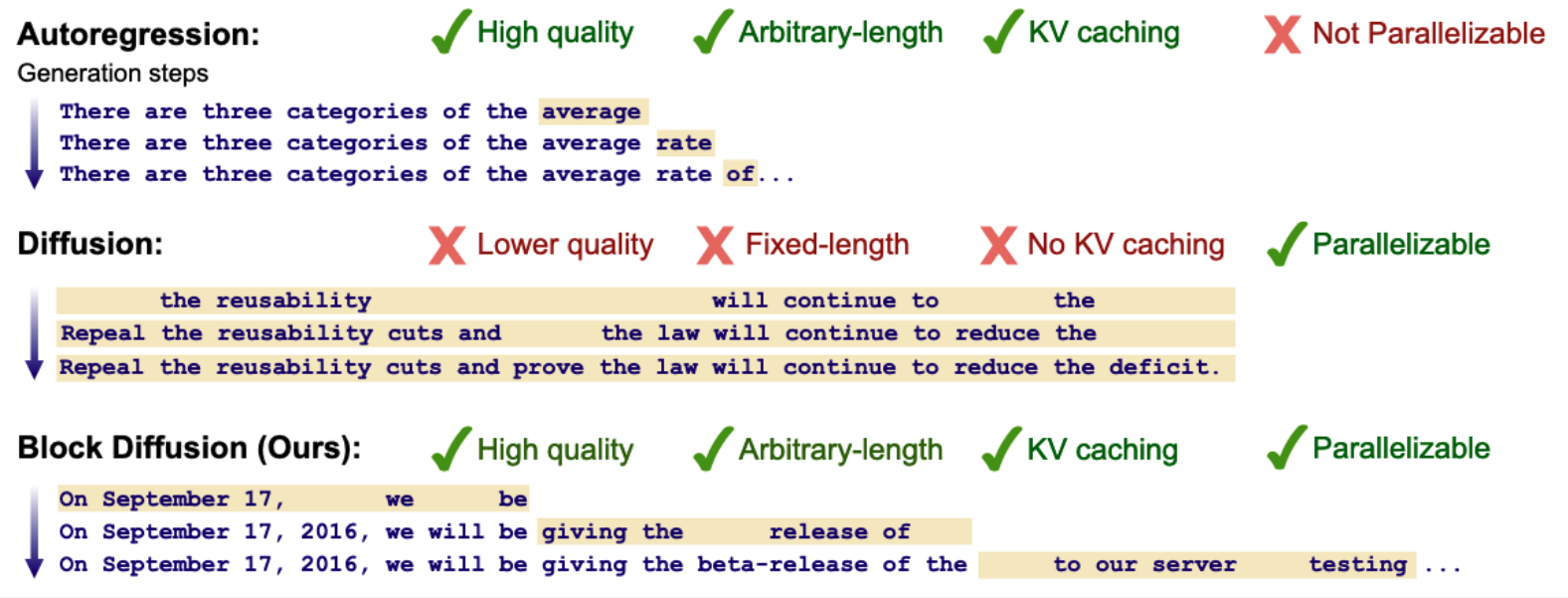
- DLM decoding time complexity
  - Transformer w/o KV caching: O(n^2)
    - If we linearly scale up timesteps with n, we get O(n^3)
  - Keeping DLM decoding time complexity to O(n^2) requires either
    - O(1) parallel decoding
    - KV cache of only O(1) tokens are re-computed (“dKV-Cache-Greedy”, Ma+ ‘25)

Ma et al., 2025, dKV-Cache: The Cache for Diffusion Language Models

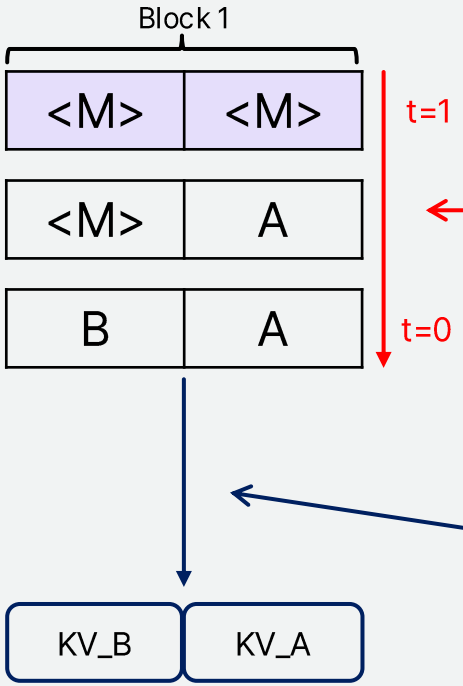
Block diffusion:
hybrid of diffusion and AR

Block diffusion

- Block diffusion(a.k.a. BD3-LM) (ICLR '25 Oral)
- Hybrid of diffusion and AR
  - Diffusion within each block
  - AR across the blocks



Arriola et al., 2025, Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models



1st forward pass:
For denoising block 1

- Input: masked tokens [<M>, <M>]
- Output: denoised tokens [B, A]

2nd forward pass:
For KV caching block 1

- Input: clean tokens [B, A]
- Output: KV of [B, A] via bidirectional attention

Algorithm 2 Block Diffusion Sampling

Input: # blocks B, model $\mathbf{x}_\theta$, diffusion sampling algorithm SAMPLE

$\mathbf{x}, \mathbf{K}, \mathbf{V} \leftarrow \emptyset$ ▷ output & KV cache

for $b = 1$ to B **do**

$\mathbf{x}^b \leftarrow \text{SAMPLE}(\mathbf{x}_\theta^b, \mathbf{K}^{1:b\text{-}1}, \mathbf{V}^{1:b\text{-}1})$

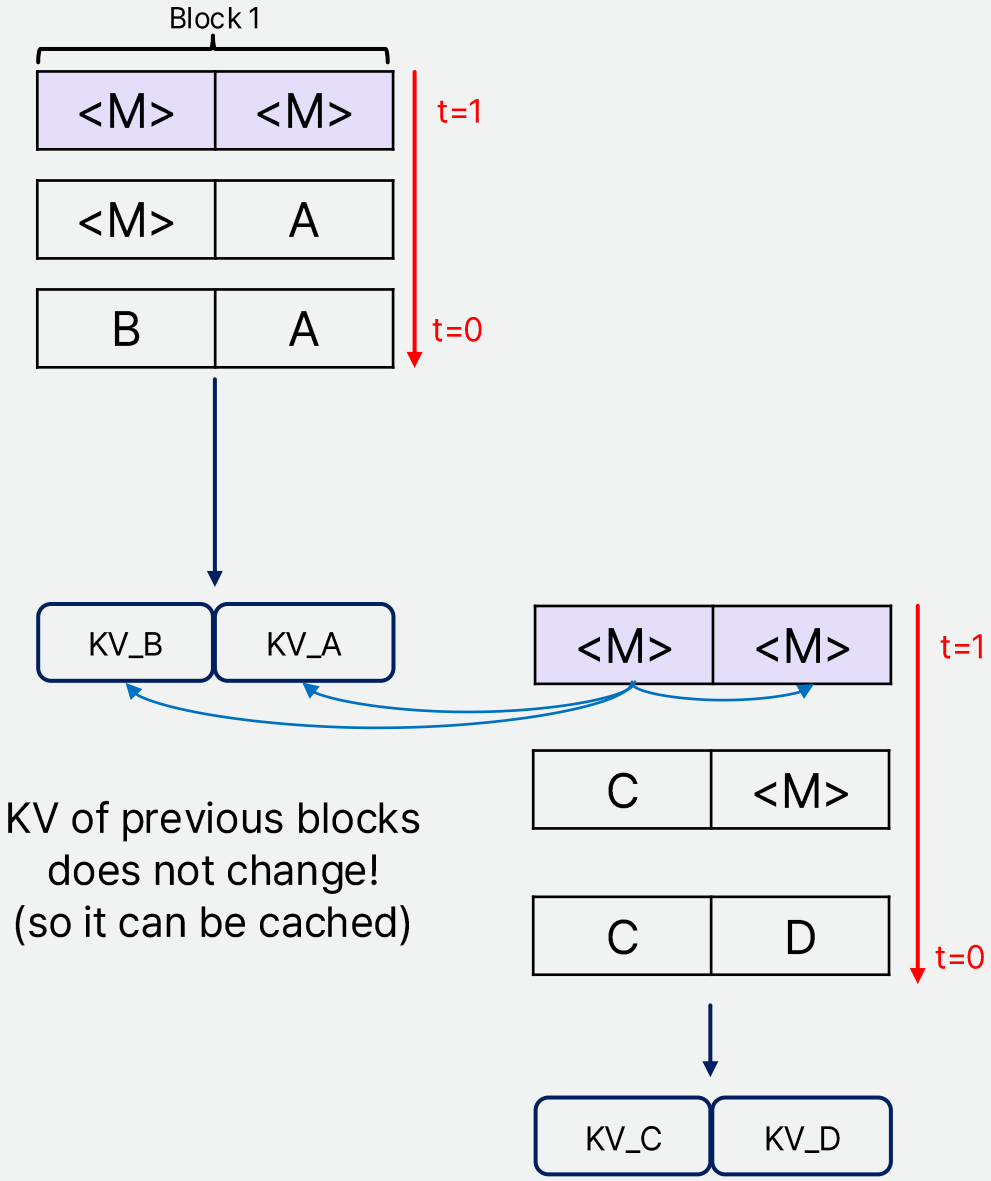
$\emptyset, \mathbf{K}^b, \mathbf{V}^b \leftarrow \mathbf{x}_\theta^b(\mathbf{x}^b)$

$\mathbf{x} \leftarrow \mathbf{x}^{1:b-1} \oplus \mathbf{x}^b$

$(\mathbf{K}, \mathbf{V}) \leftarrow (\mathbf{K}^{1:b-1} \oplus \mathbf{K}^b, \mathbf{V}^{1:b-1} \oplus \mathbf{V}^b)$

end for

return $\mathbf{x}$



Algorithm 2 Block Diffusion Sampling

Input: # blocks B, model $\mathbf{x}_\theta$, diffusion sampling algorithm SAMPLE

$\mathbf{x}, \mathbf{K}, \mathbf{V} \leftarrow \emptyset$ ▷ output & KV cache

for $b = 1$ to B **do**

$\mathbf{x}^b \leftarrow \text{SAMPLE}(\mathbf{x}_\theta^b, \mathbf{K}^{1:b\text{-}1}, \mathbf{V}^{1:b\text{-}1})$

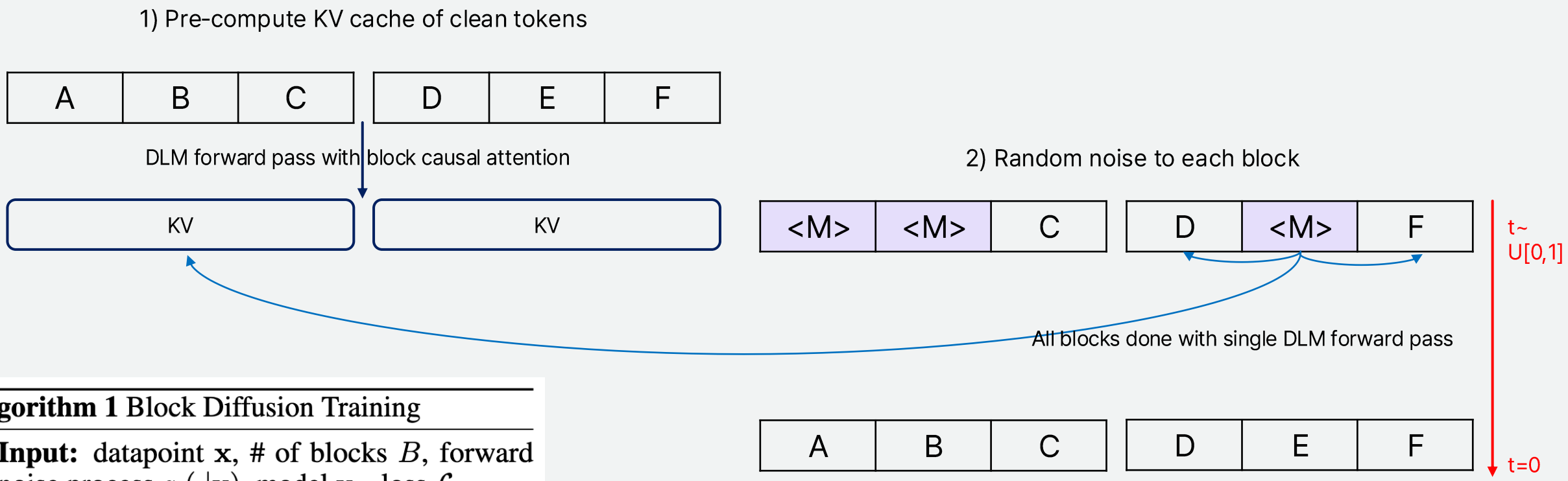
$\emptyset, \mathbf{K}^b, \mathbf{V}^b \leftarrow \mathbf{x}_\theta^b(\mathbf{x}^b)$

$\mathbf{x} \leftarrow \mathbf{x}^{1:b-1} \oplus \mathbf{x}^b$

$(\mathbf{K}, \mathbf{V}) \leftarrow (\mathbf{K}^{1:b-1} \oplus \mathbf{K}^b, \mathbf{V}^{1:b-1} \oplus \mathbf{V}^b)$

end for

return $\mathbf{x}$



Algorithm 1 Block Diffusion Training

Input: datapoint $\mathbf{x}$, # of blocks B, forward noise process $q_t(\cdot|\mathbf{x})$, model $\mathbf{x}_\theta$, loss $\mathcal{L}_{\text{BD}}$

repeat

Sample $t_1, \ldots, t_B \sim \mathcal{U}[0, 1]$

$\forall b \in \{1, ..., B\} : \mathbf{x}_{t_b}^b \sim q_{t_b}(\cdot|\mathbf{x}^b)$

$\emptyset, \mathbf{K}^{1:B}, \mathbf{V}^{1:B} \leftarrow \mathbf{x}_\theta(\mathbf{x})$ ▷ KV cache

$\forall b$: $\mathbf{x}_{\text{logit}}^b, \emptyset, \emptyset \leftarrow \mathbf{x}_\theta^b(\mathbf{x}_{t_b}^b, \mathbf{K}^{1:b\text{-}1}, \mathbf{V}^{1:b\text{-}1})$

Let $\mathbf{x}_{\text{logit}} \leftarrow \mathbf{x}_{\text{logit}}^1 \oplus \cdots \oplus \mathbf{x}_{\text{logit}}^B$

Take gradient step on $\nabla_\theta \mathcal{L}_{\text{BD}}(\mathbf{x}_{\text{logit}}; \theta)$

until converged

Block diffusion is very much like AR

- With below two constraints, block diffusion is equivalent to AR (in expectation)
  - Block size = 1
  - Tuned schedule: During training, always mask the tokens of each block, i.e., $q(\mathbf{x}_t^l = \mathbf{m} | \mathbf{x}^l) = 1$

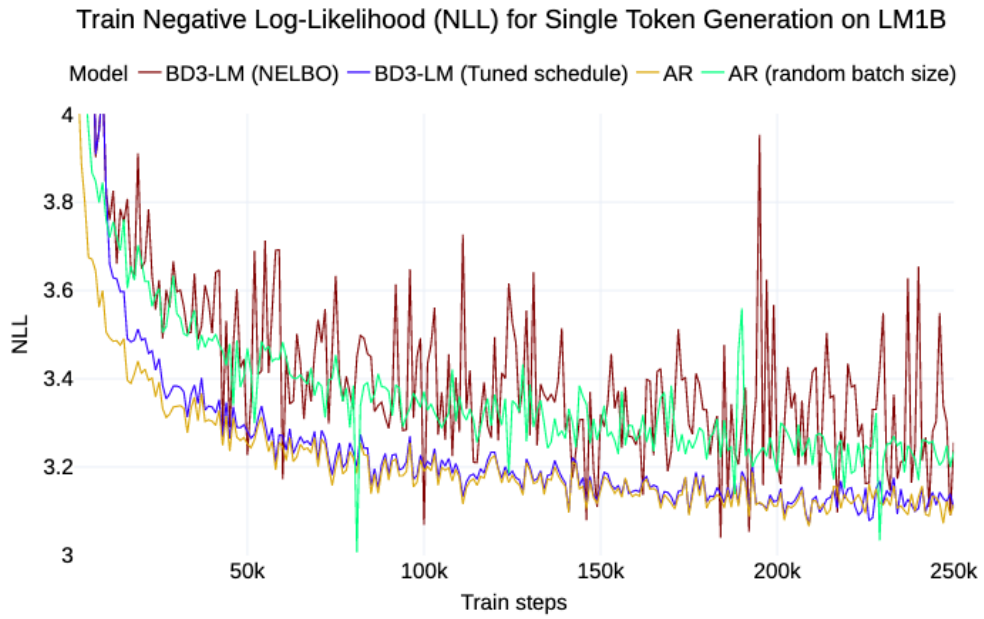


Figure 2: Train NLLs for modeling the per-token likelihood on LM1B. Models are trained on 16B tokens. Training under the discrete diffusion NELBO, where half of the tokens in a batch are masked on average, has similar training variance to an AR model with a random batch size.

Vanilla BD3-LM and AR with random batch size show higher variance

(both shows inconsistent #(unmasked tokens) per step)

NLL curve of BD3-LM (Tuned schedule) == AR

Block diffusion -- results

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

L′ = block size

context_length = 128

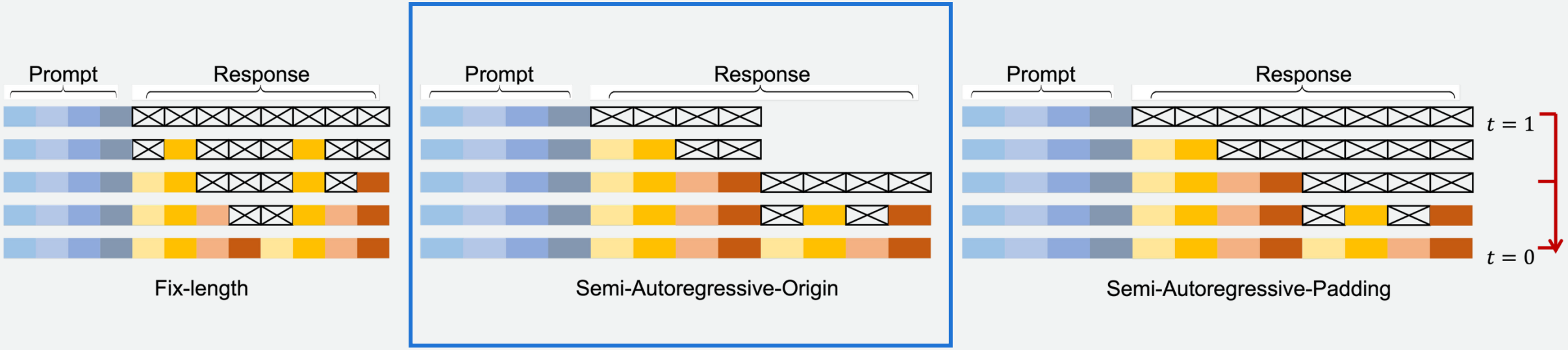
Table 4: Test perplexities (PPL; ↓) on OWT for models trained for 524B tokens. Best diffusion value is bolded.

	PPL (↓)
AR (Sahoo et al., 2024a)	17.54
SEDD (Lou et al., 2024)	$\leq$ 24.10
MDLM (Sahoo et al., 2024a)	$\leq$ 22.98
BD3-LMs $L' = 16$	$\leq$ 22.27
$L' = 8$	$\leq$ 21.68
$L' = 4$	$\leq$ **20.73**

context_length = 1024

Now you can differentiate between

- LLaDA's semi-AR sampling *vs* block diffusion's semi-AR sampling
- Intra-block (approximate) KV caching *vs* inter-block (exact) KV caching



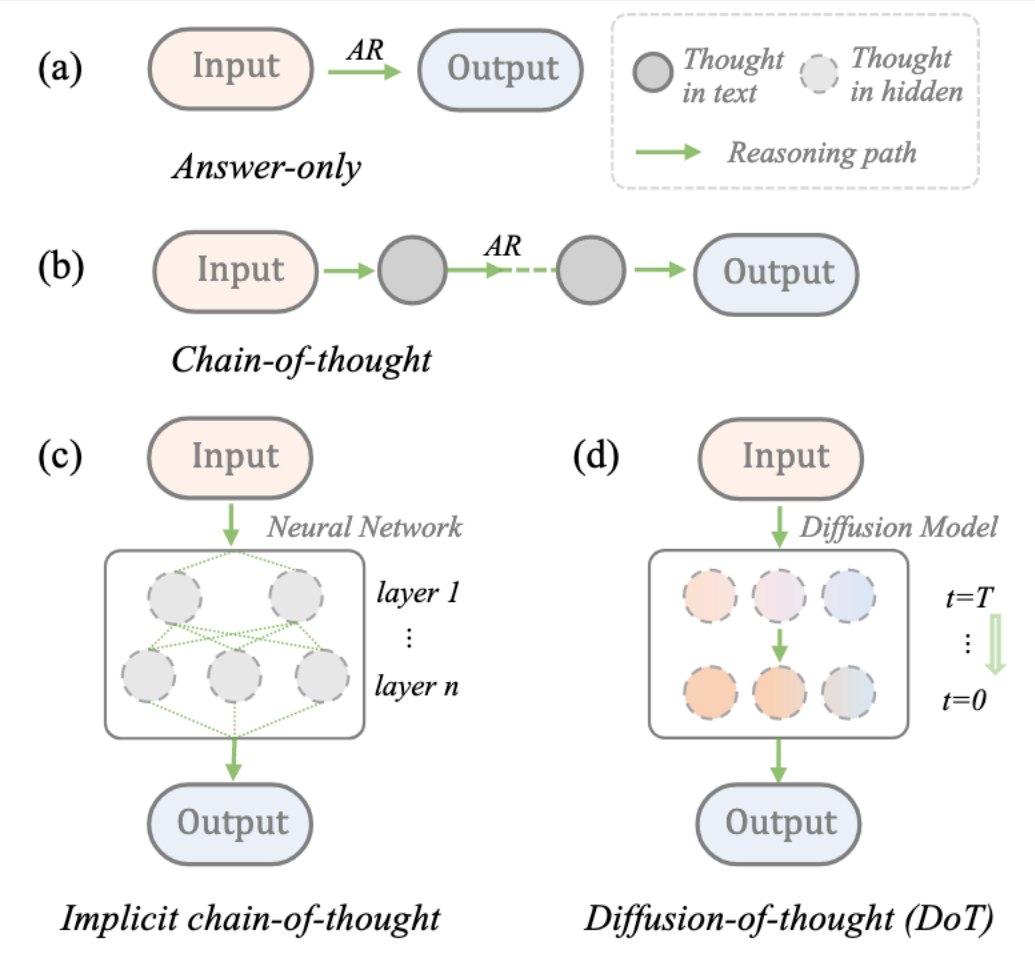
https://github.com/ML-GSAI/LLaDA/blob/main/GUIDELINES.md

Post-training

Diffusion of Thought (NeurIPS '24)

- AR LMs use chain-of-thought to solve hard problems. How can DLMs use similar approach?

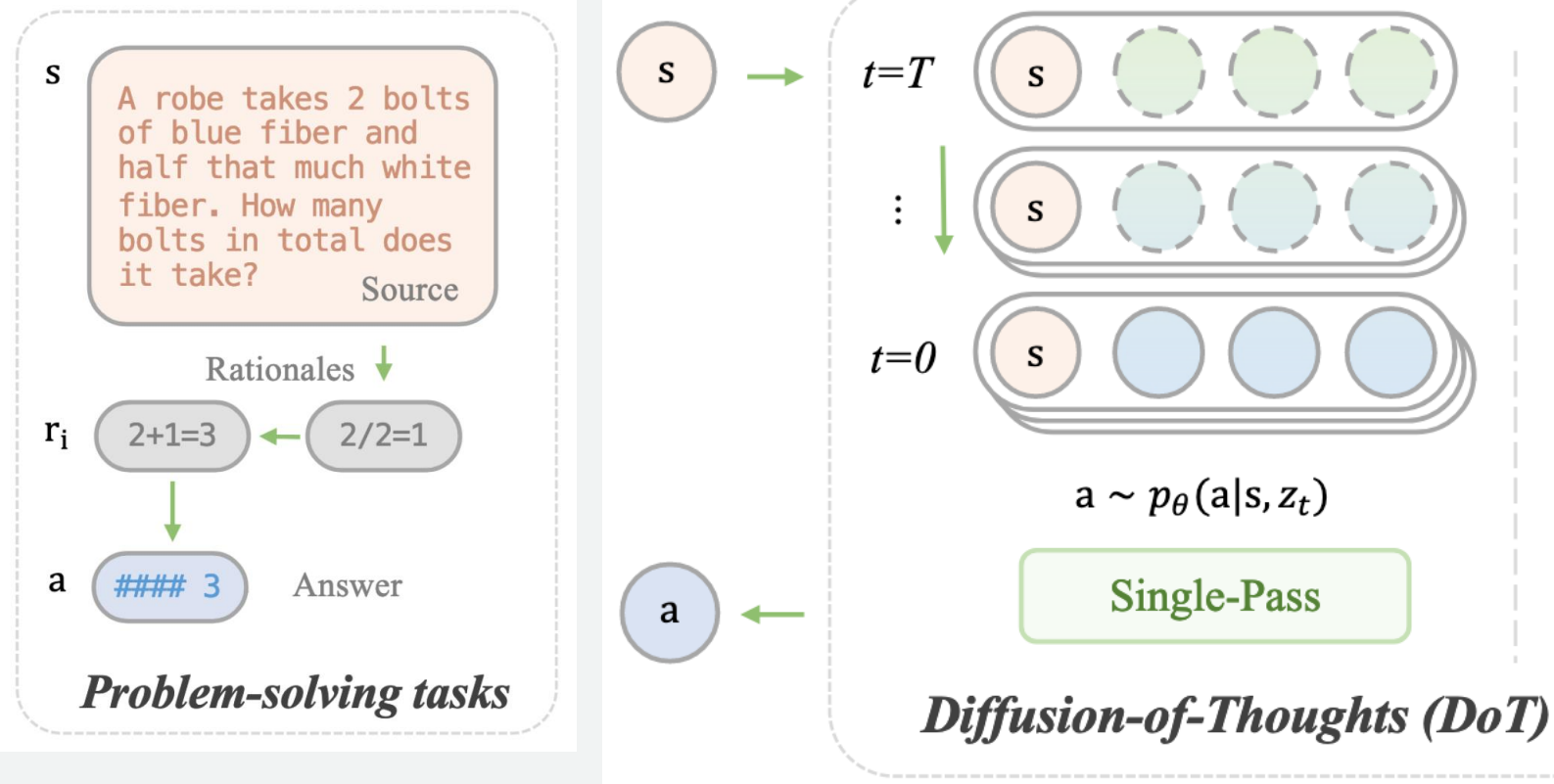
- Diffusion-of-thought(DoT) generates reasoning path along the diffusion timesteps



Ye et al., 2024, Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models

DoT -- modeling

Only the rationale tokens are noised

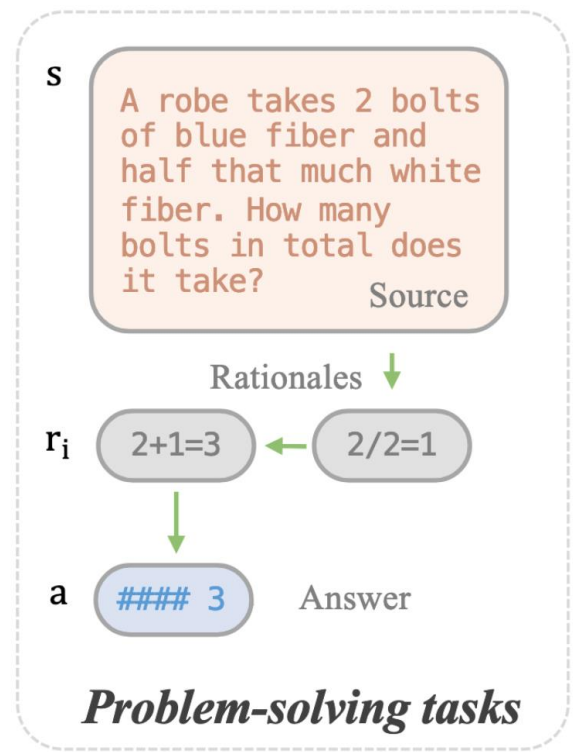


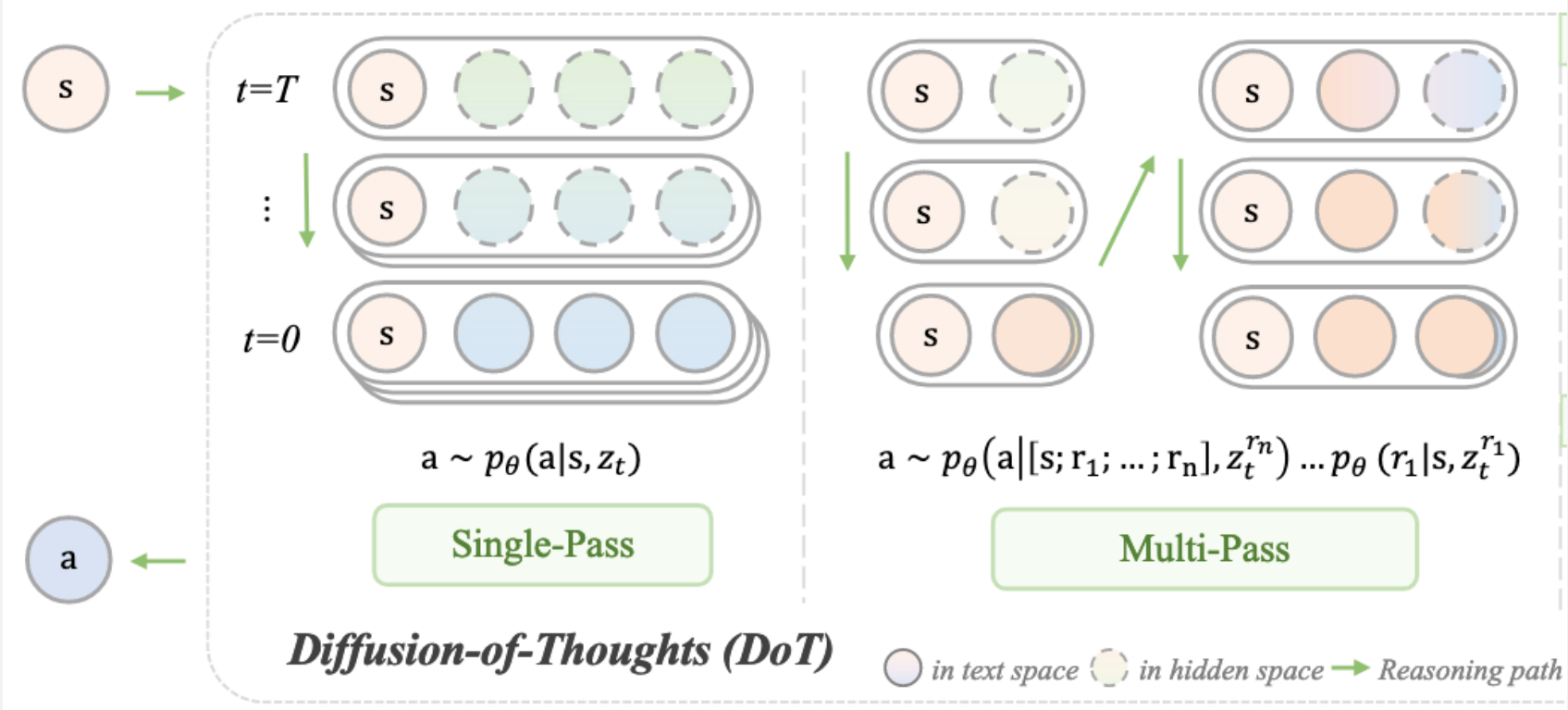
All rationales are generated in parallel

Ye et al., 2024, Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models

DoT -- modeling

Multi-pass DoT == semi-AR version

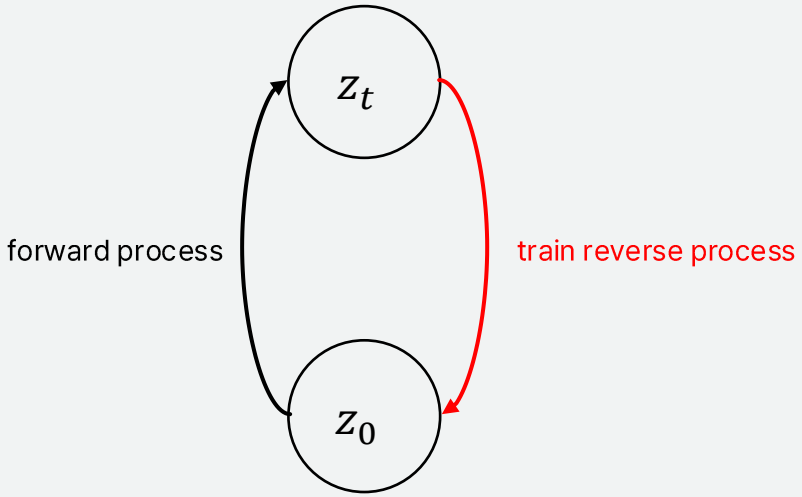




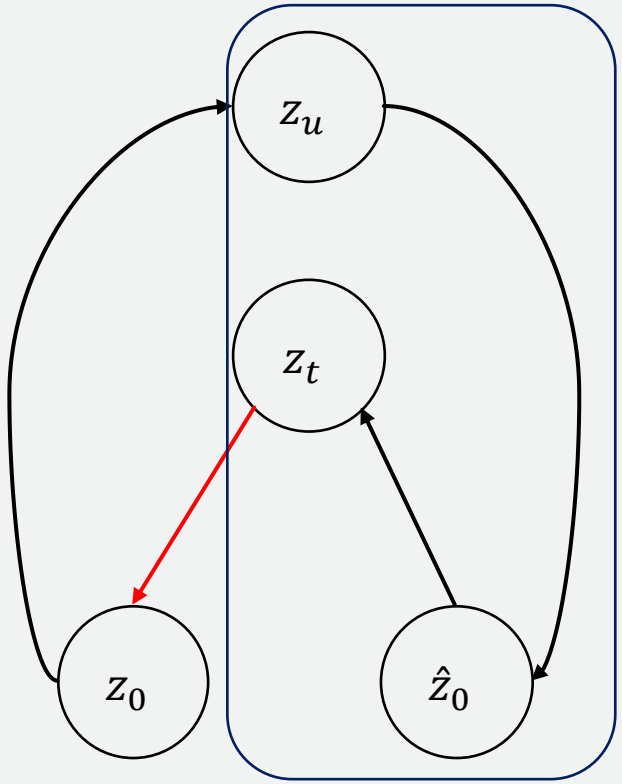
Ye et al., 2024, Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models

Training DoT

- Scheduled sampling helps the model to recover from its reasoning errors



Without scheduled sampling



With scheduled sampling

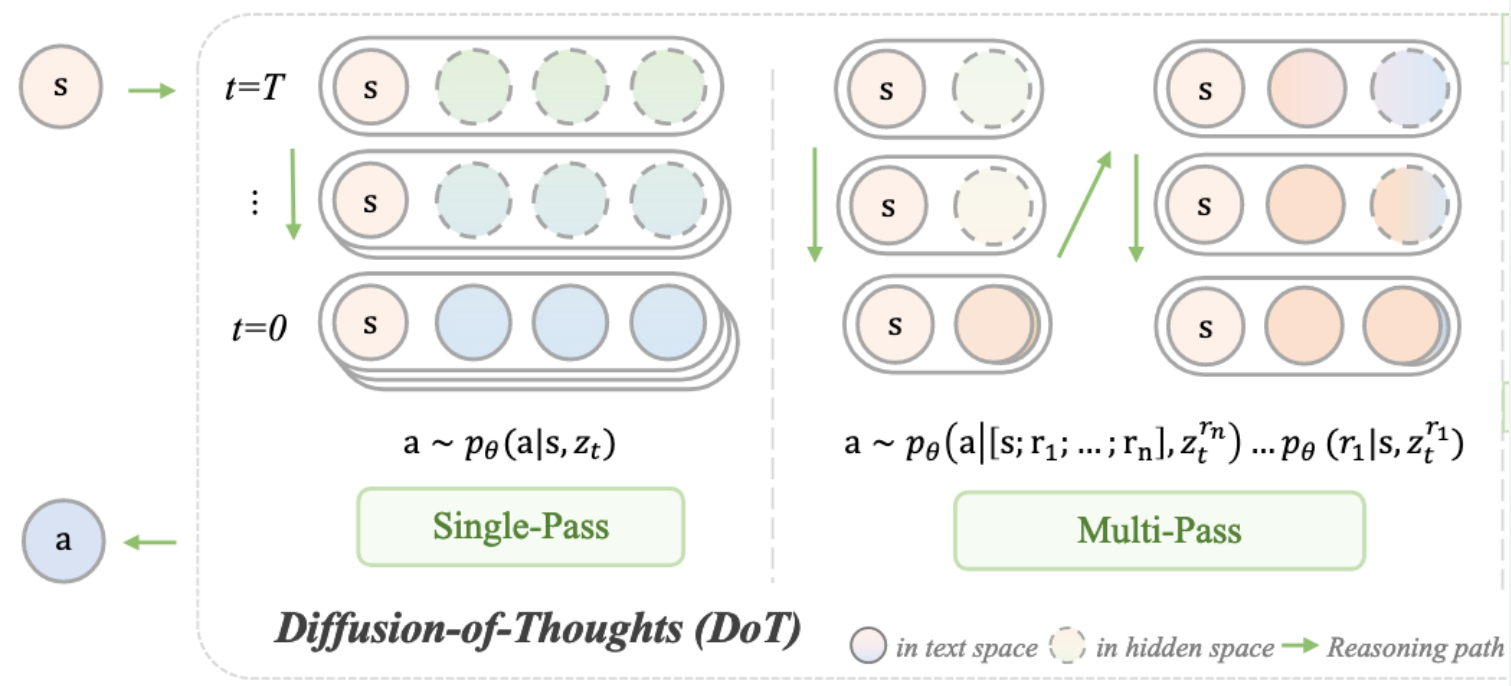
- Sample from z_u with prob $\epsilon_i \in [1, \epsilon_{min}]$, i is a training step and ϵ_i linearly decays

(Authors suggest $\epsilon_{min} = .95$)

- $u \sim U(t+1, T)$

Training DoT-MP

- Coupled sampling helps the model to recover from its reasoning errors
  - Add noise to not only the current block but also to the previous blocks



Ye et al., 2024, Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models

DoT -- result

- DoT-MP (semi-AR) was better than DoT (non-AR)
  - Coupled sampling helps in DoT-MP
- GSM8K(math) favors left-to-right ordering, but for other tasks, semi-AR sampling may not have an edge
  - Coding?

Models	Acc. (%) ↑
Continue pre-training	0.5
DoT-finetune	32.6
(-) scheduled sampling	31.2
DoT^{MP}-finetune	37.7
(-) coupled sampling	35.5

Table 2: Ablation of Plaid DoT on GSM8K.

Post-training DLMs

- DoT is the first work to present post-training for DLMs
  - DoT = SFT on CoT dataset

- In RL, applying GRPO for DLMs is challenging because obatining the log-likelihood of tokens (thereby sequences) is hard and inefficient
  - In contrast, applying PPO seems to be more straightforward?

Estimating the likelihood of each token is hard in DLMs

There are so many paths to decode a single token. “Sampling” shows just one of many.

<-> In AR, there is one and only path to decode a single token.

<M>	<M>	<M>	<M>	<M>	<M>

<M>	<M>	<M>	<M>	<M>	F

<M>	<M>	<M>	<M>	<M>	<M>

.
.

A	B	C	D	E	<M>

A	B	C	D	E	F

How to efficiently estimate

- LLaDA uses Monte Carlo estimation with random maksing on response tokens
  - c.f. LLaDA does not use GRPO, but algorithm for log-likelihood estimation is suggested

Algorithm 3 Conditional Log-likelihood Evaluation of LLaDA

Require: mask predictor p_θ, prompt p_0, response r_0, the number of Monte Carlo estimations n_{mc}

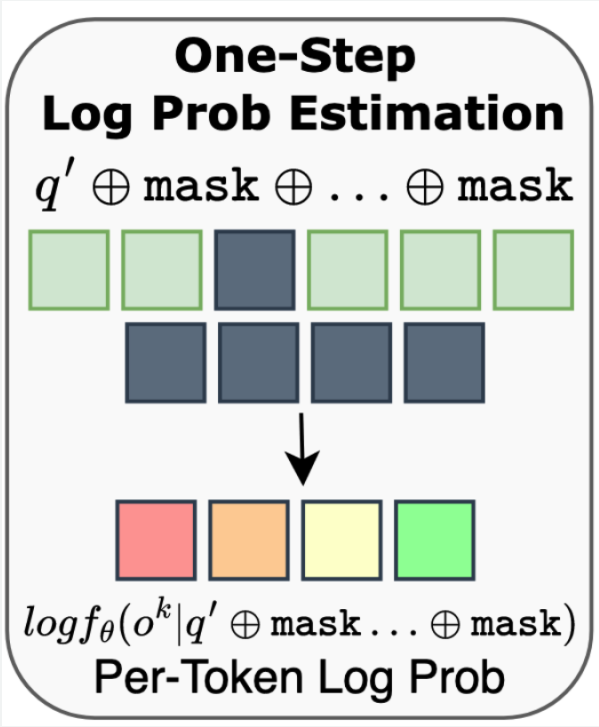
log_likelihood $= 0$
for $i \leftarrow 1$ to n_{mc} **do**
  $l \sim \{1, 2, \ldots, L\}$ # L is the sequence length of r_0
  Obtain r_l by uniformly sampling l tokens from r_0 without replacement for masking
  log_likelihood $=$ log_likelihood $+ \frac{L}{l} \sum_{i=1}^{L} \mathbf{1}[r_l^i = \mathrm{M}] \log p_\theta(r_0^i | p_0, r_l)$
end for
log_likelihood $=$ log_likelihood$/n_{mc}$
Return log_likelihood

- Problem? Monte Carlo estimation is too costly

d1: estimation via one-step

- Zhao et al., 2025, d1: Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning

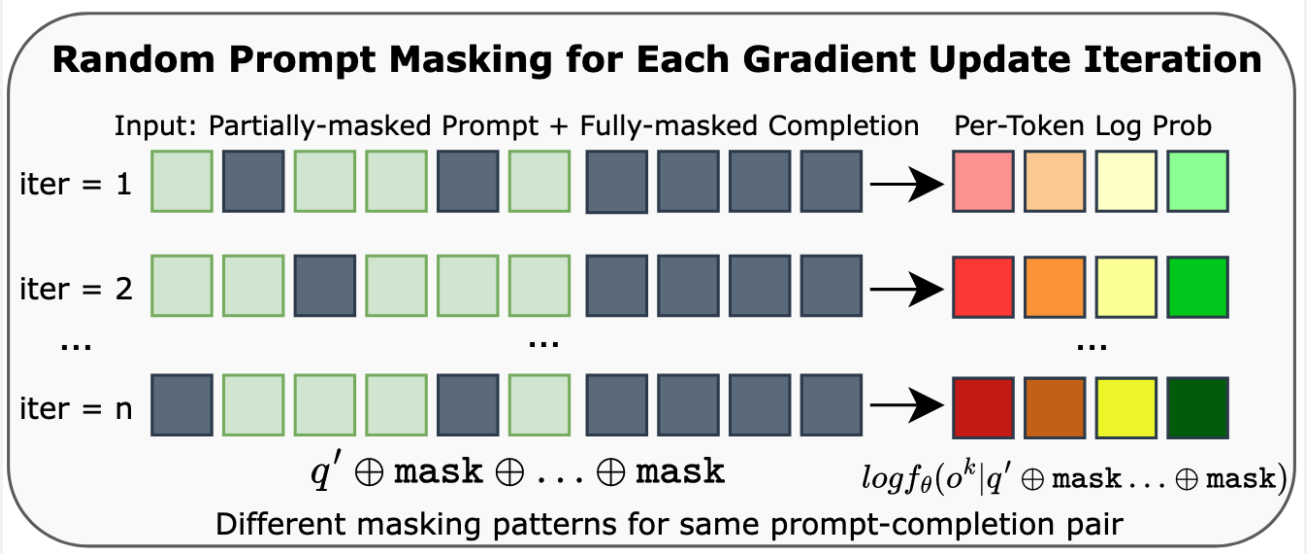
- d1 proposes *diffu-GRPO*, where log-likelihood of all tokens are estimated by a single unmasking process, i.e., t = T -> t =0
- Log-likelihood of the sequence is simply a sum of the log-likelihood of all tokens



Mask the prompt?

- d1 also masks 15% of prompt tokens
  - d1 claims that this serves as a regularization for policy optimization

- This is later rejected by DiffuCoder (Gong+ ‘25)
  - DiffuCoder reports no yield in stable reward improvement
  - Difference: math(d1) *vs* coding(DiffuCoder)



Zhao et al., 2025, d1: Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning

References

- Sampling
  - Nie et al., 2025, Large Language Diffusion Models
  - Wu et al., 2025, Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding
  - Ma et al., 2025, dKV-Cache: The Cache for Diffusion Language Models
  - Arriola et al., 2025, Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models
- Post-training
  - Ye et al., 2024, Diffusion of Thought: Chain-of-Thought Reasoning in Diffusion Language Models
  - Zhao et al., 2025, d1: Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning
  - Gond et al., 2025, DiffuCoder: Understanding and Improving Masked Diffusion Models for Code Generation

Thank you!

Yejoon Lee
leeyejoon@snu.ac.kr

AIDAS Lab